



CLARITY ENGINE

Plateforme SaaS de Gouvernance de Marque





TABLE DES MATIÈRES

- 1. Présentation générale du projet 3
- 2. Objectifs du projet 3
 - 2.1 Objectif principal 3
 - 2.2 Objectifs fonctionnels 3
 - 2.3 Objectifs techniques 4
- 3. Périmètre du projet (Scope) 4
 - 3.1 Fonctionnalités dans le périmètre 4
 - 3.2 Fonctionnalités hors périmètre 5
- 4. Architecture applicative 5
 - 4.1 Modèle client-serveur 3 niveaux 5
 - 4.2 Flux principal d'une analyse 6
- 5. Technologies utilisées 6
 - 5.1 Backend (Python / FastAPI) 6
 - 5.2 Frontend (React / TypeScript) 7
 - 5.3 Infrastructure & outils de dev. 7
- 6. Schéma de la base de données 8
- 7. Moteur d'évaluation IA 8
- 8. Sécurité et déterminisme 9

1

PRÉSENTATION GÉNÉRALE

Vue d'ensemble de la plateforme et de son contexte

Contexte & Enjeux

Clarity Engine est une plateforme SaaS de **gouvernance de marque** développée lors d'un stage au sein de **Zone Bleue**, agence de communication stratégique. La plateforme permet d'évaluer automatiquement des messages de communication — communiqués de presse, publications LinkedIn, e-mails institutionnels, discours — en les confrontant à un référentiel structuré appelé **Brand System**.

1

SaaS

Modèle de livraison

5

dims

Dimensions d'évaluation

6

langues

Couverture multilingue

JWT

Authentification sécurisée

Contrairement à un chatbot ou générateur de contenu générique, Clarity Engine est un **moteur d'évaluation de précision** : chaque analyse produit un score chiffré (/100), cinq sous-scores dimensionnels, un niveau de risque narratif (*faible / modéré / élevé*), ainsi que des recommandations d'amélioration ancrées dans les lignes éditoriales propres à chaque marque.

Note : La plateforme ne génère pas de communication ex nihilo. Elle évalue et améliore des messages existants dans le respect strict du Brand System associé.

2

OBJECTIFS DU PROJET

Objectif principal, fonctionnels et techniques

2.1 Objectif principal

Automatiser et **objectiver l'évaluation de la cohérence de marque** dans les productions écrites, en remplaçant une revue manuelle subjective par une analyse structurée, reproductible et entièrement documentée.

2.2 Objectifs fonctionnels

#	Objectif	Description
O 1	Évaluation automatisée	Analyser tout message contre un Brand System → score /100 + 5 sous-scores.
O 2	Gestion des Brand Systems	Création, édition versionnée et consultation de référentiels multi-clients.
O 3	Réécriture assistée	Version améliorée du message + liste des corrections effectuées.
O 4	Historique & reporting	Conservation de toutes les analyses + statistiques agrégées.
O 5	Multi-utilisateurs & multi-rôles	Distinction admins (accès global) / clients (périmètre restreint).
O 6	Import de documents	Upload PDF/texte → pipeline Celery async → consultation du statut.

2.3 Objectifs techniques

- Architecture découplée (frontend / backend / BDD / IA externe) pour l'évolutivité.
- Évaluations **déterministes et reproductibles** : temperature=0, seed=0 sur tous les appels LLM.
- Authentification JWT sécurisant l'intégralité des routes sensibles.
- API REST documentée et consommable par n'importe quel client HTTP.



PÉRIMÈTRE DU PROJET

In-Scope & Out-of-Scope

3.1 Fonctionnalités dans le périmètre (In-Scope)

Fonctionnalité	Description
Authentification	Inscription, connexion, JWT, déconnexion.
Gestion Brand Systems	Création, édition versionnée, désactivation logique.
Moteur IA	Appel Groq, parsing JSON, retry ×3, normalisation des scores.
5 dimensions d'évaluation	Clarté, Alignement, Focus, Ton, Contribution narrative.
Réécriture assistée	Génération version améliorée + liste des modifications.
Tableau de bord	KPIs globaux, distribution des risques, analyses récentes.
Historique	Filtrage par risque, canal, période de dates.
Panel Admin	Vue globale clients, Brand Systems, toutes analyses.
Import documents	Upload → Celery async → consultation du statut.
Multi-langue	FR, EN, ES, DE, AR, PT.

3.2 Fonctionnalités hors périmètre (Out-of-Scope)

Fonctionnalité	Justification
Génération ex nihilo	Clarity Engine améliore ; il ne génère pas depuis un brief vide.
Réseaux sociaux	Aucun connecteur LinkedIn, Twitter ou autre prévu.
Analyse visuelle	Le moteur traite uniquement du texte.
Déploiement cloud	Opéré localement ; AWS/GCP/Azure hors périmètre.
Fine-tuning LLM	Llama 3.3 utilisé tel quel via l'API Groq.
Facturation	L'aspect commercial et monétisation est hors périmètre.
Application mobile	Interface web uniquement ; pas d'app iOS/Android native.

4

ARCHITECTURE APPLICATIVE

Modèle client-serveur 3 niveaux & flux d'analyse

4.1 Modèle client-serveur à 3 niveaux

L'architecture suit un modèle **client-serveur à 3 niveaux** strictement découplés. Le frontend React (SPA) communique exclusivement avec le backend FastAPI via une API REST sécurisée par JWT. Le backend orchestre deux dépendances principales : la base de données SQLite pour la persistance, et l'API Groq pour l'intelligence artificielle. Un composant optionnel Celery + Redis gère les traitements asynchrones (import de documents volumineux).

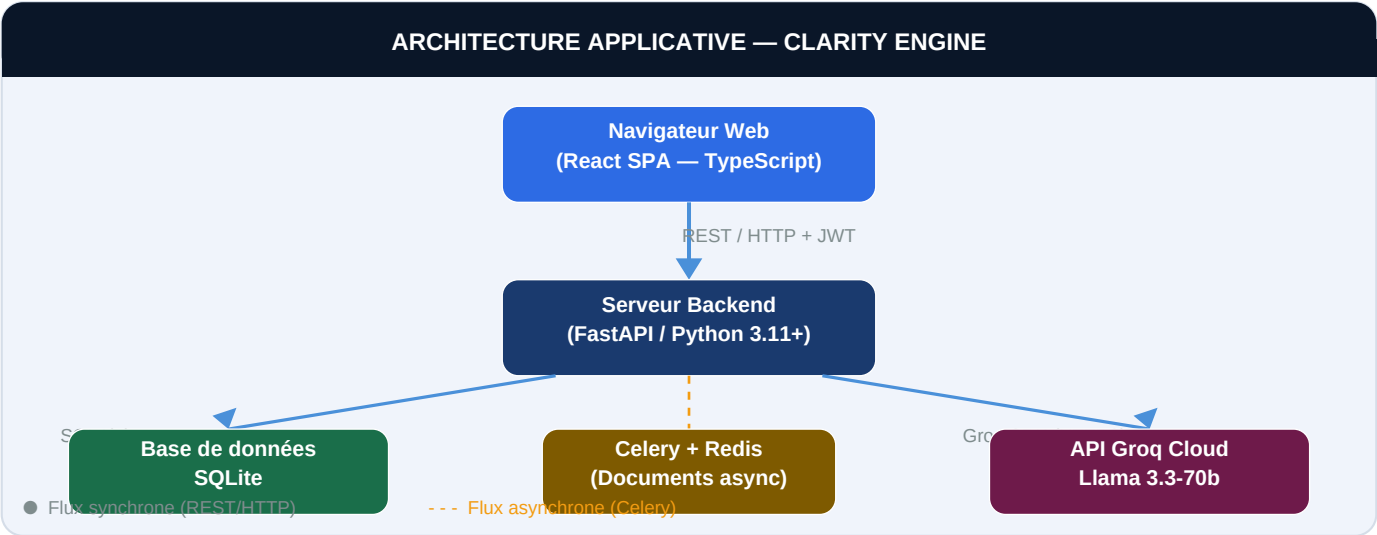


Figure 1 — Diagramme d'architecture applicative de Clarity Engine. Les traits pleins représentent les flux synchrones (REST/HTTP) ; les tirets représentent les flux asynchrones (Celery/Redis).

4.2 Flux principal d'une analyse

#	Étape	Description
1	Saisie utilisateur	L'utilisateur sélectionne un Brand System et saisit son message dans l'interface React.
2	Requête HTTP	Le frontend envoie POST /api/analyze avec le token JWT en header Authorization.
3	Récupération BDD	Le backend charge le Brand System depuis SQLite via SQLAlchemy.
4	Construction du prompt	Assemblage du prompt structuré : directives Brand System + message à analyser.
5	Appel LLM	Appel à l'API Groq (Llama 3.3-70b, temperature=0, seed=0) → réponse JSON.
6	Parsing & validation	Parsing JSON, validation Pydantic, 3 tentatives max, normalisation des scores (0–100).
7	Persistance	Sauvegarde de l'analyse complète en base de données SQLite.
8	Affichage résultats	Le frontend reçoit l'identifiant, redirige vers la page de résultats détaillés.

Tableau 2 — Flux séquentiel d'une analyse de message via Clarity Engine.

5

TECHNOLOGIES UTILISÉES

Stack complète Backend, Frontend & Infrastructure

5.1 Backend — Python / FastAPI

Technologie / Version	Rôle
FastAPI 0.136	Framework API REST, validation Pydantic, cycle de vie async.
Uvicorn 0.46	Serveur ASGI asynchrone, exécution du processus FastAPI.
SQLAlchemy 2.0	ORM — mapping objet-relationnel, requêtes via sessions.
Alembic 1.18	Gestion versionnée des migrations de schéma BDD.
SQLite	Base de données relationnelle embarquée (fichier clarity.db).
python-jose	Génération et vérification des tokens JWT.
bcrypt / passlib	Hachage sécurisé unidirectionnel des mots de passe.
Groq SDK 1.2	Client Python officiel pour l'API Groq (LLM Llama 3.3-70b).
Celery 5.6	File de tâches distribuées — traitement asynchrone de documents.
Redis 7.4	Broker de messages pour Celery.
pypdf 6.10	Extraction du contenu textuel des PDF uploadés.

5.2 Frontend — React / TypeScript

Technologie / Version	Rôle
React 19.2	Bibliothèque de composants pour l'interface réactive (SPA).
TypeScript ~6.0	Typage statique, sécurité à la compilation, maintenabilité.
Vite 8.0	Bundler ultra-rapide avec Hot Module Replacement (HMR).
react-router-dom 7	Gestion des routes côté client (navigation SPA).
Recharts 3.8	Graphiques (barres, camemberts) pour scores et statistiques.
Tailwind CSS 4.2	Classes utilitaires CSS en complément du design system.
Lora + DM Sans	Typographie éditoriale : titres (Lora) + corps (DM Sans).
Fetch API (natif)	Requêtes HTTP vers le backend avec token JWT en header.

5.3 Infrastructure & outils de développement

Outil	Rôle
Git	Versioning du code source, gestion des branches de développement.
Fichier .env	Stockage des secrets : clé API Groq, clé secrète JWT, URL BDD.
venv (Python)	Isolation des dépendances Python du projet.

Outil	Rôle
npm	Gestion des dépendances JavaScript du frontend.
Local / Windows + macOS	Environnement cible de développement et de tests.

5.4 API externe

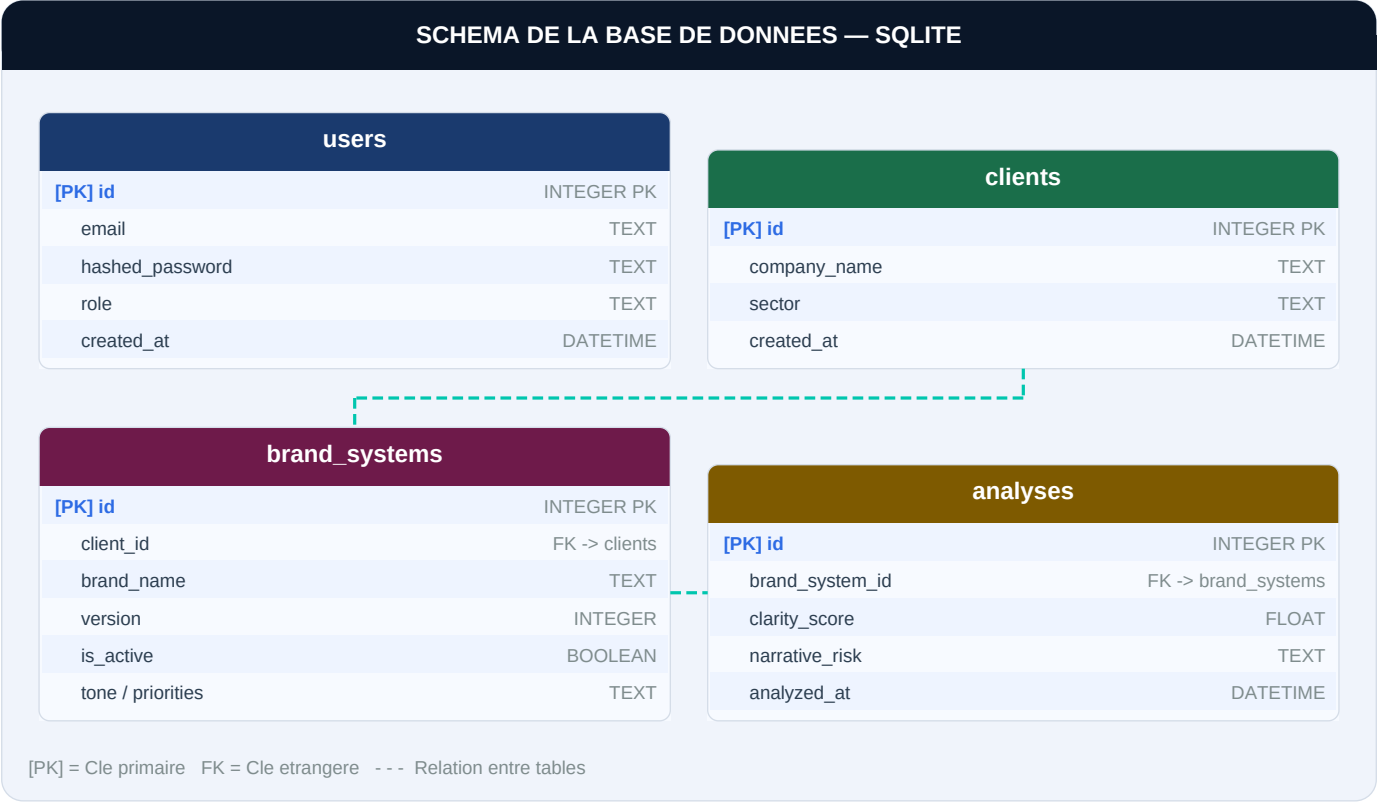
Service	Description
Groq API (cloud)	Service d'inférence LLM haute performance. Reçoit un prompt structuré (Brand System + message) et retourne une évaluation JSON.
Llama 3.3-70b-versatile	LLM open source de Meta, 70 milliards de paramètres. temperature=0, seed=0 pour garantir le déterminisme des sorties.

6

SCHÉMA DE LA BASE DE DONNÉES

Structure relationnelle SQLite — 4 tables principales

La base de données SQLite comprend **4 tables principales** liées par des clés étrangères. La table *brand_systems* est versionnée : chaque modification incrémente le champ version sans effacer l'historique.





MOTEUR D'ÉVALUATION IA

Dimensions, scoring et mécanisme de retry

Les 5 dimensions d'évaluation

Chaque message est évalué selon **5 dimensions indépendantes**, chacune notée sur 20 points, pour un total de **100 points (Clarity Score)**. Un niveau de risque narratif — *faible*, *modéré* ou *élevé* — est également calculé en fonction du score global et des faiblesses identifiées.

Dimension	Description	Pondération
Clarté linguistique	Évalue la lisibilité, la précision lexicale et l'absence d'ambiguïté syntaxique du message.	20 pts
Alignement stratégique	Mesure la cohérence entre le message et le positionnement, la mission et les priorités de la marque.	20 pts
Focus	Détermine la concentration du propos : un seul message central, sans dilution ni surcharge informationnelle.	20 pts
Ton	Évalue la conformité du registre émotionnel et stylistique avec les directives tonales du Brand System.	20 pts
Contribution narrative	Apprécie la contribution du message à l'histoire globale de la marque et à sa cohérence temporelle.	20 pts

Tableau 5 — Les 5 dimensions d'évaluation du moteur Clarity Engine.

Mécanisme de robustesse (retry)

- **Tentative 1** : appel à l'API Groq avec le prompt complet.
- **Tentative 2** (si erreur de parsing) : reformulation du prompt avec instructions JSON renforcées.
- **Tentative 3** (si nouvelle erreur) : dernier appel avec schéma JSON explicite en exemple.
- **Échec définitif** : l'analyse est marquée comme échouée et une erreur métier est remontée.

Après réception d'une réponse valide, les scores sont **normalisés** (clamping entre 0 et 20 par dimension, 0 et 100 globalement) par une fonction dédiée afin de garantir l'intégrité des données persistées.

Format de sortie JSON

POST /api/analyze → réponse structurée :

```
{ "clarity_score": 82, "sub_clarity": 17, "sub_alignment": 16, "sub_focus": 18, "sub_tone": 15, "sub_narrative_contribution": 16, "narrative_risk": "faible", "points_forts": [...], "points_faibles": [...], "recommandations": [...], "rewritten_message": "..."} 
```



SÉCURITÉ & DÉTERMINISME

Authentification JWT, hachage et reproductibilité

Authentification JWT

- Toutes les routes sensibles sont protégées par un middleware de vérification JWT.
- Les tokens sont générés côté backend (python-jose) à la connexion et transmis dans le header Authorization: Bearer <token>.
- Les mots de passe sont hachés avec bcrypt (via passlib) — stockage unidirectionnel, sans possibilité de déchiffrement.
- Séparation des rôles : **admin** (accès global) vs **client** (périmètre restreint à sa marque).

Déterminisme des évaluations

Garantir qu'un même message soumis au même Brand System produise **toujours les mêmes scores** est une contrainte fonctionnelle fondamentale de Clarity Engine. Elle est assurée par deux paramètres fixés sur chaque appel LLM :

Paramètre	Valeur	Effet
temperature	0	Supprime toute composante aléatoire dans la génération de tokens.
seed	0	Initialise le générateur pseudo-aléatoire à une valeur fixe.

Cette contrainte de déterminisme permet notamment de réaliser des *audits de cohérence* sur des corpus de messages historiques sans que les résultats soient influencés par la variabilité intrinsèque du modèle LLM.